Lab 4 — RESTful Services & OpenAPI

Bookstore application — JSON API over the existing MVC project

Guilherme Trindade · CSD230 W26 · Branch `lab4-rest-implementation`

1. API Design

Four resources are exposed as JSON under `/api/rest/`, each with the full set of five CRUD operations. All requests and responses are `application/json`; errors come back as plain text with a meaningful status code.

Security. `/api/rest/**` is `permitAll()` with CSRF disabled for that path only, so the API can be called from curl with no session. Every HTML route keeps its form login and CSRF protection — `GET /books` without a session still answers `302 → /login`.

1.1 Books — `/api/rest/books`

Verb	Path	Expects	Returns	Errors
GET	`/api/rest/books`	—	`200` array of Book	—
GET	`/api/rest/books/{id}`	—	`200` Book	`404`
POST	`/api/rest/books`	Book	`200` Book with generated id	—
PUT	`/api/rest/books/{id}`	Book	`200` Book	`404`
DELETE	`/api/rest/books/{id}`	—	`200` empty	`404`, `409`

1.2 Magazines — `/api/rest/magazines`

Verb	Path	Expects	Returns	Errors
GET	`/api/rest/magazines`	—	`200` array of Magazine	—
GET	`/api/rest/magazines/{id}`	—	`200` Magazine	`404`
POST	`/api/rest/magazines`	Magazine	`200` Magazine with generated id	—
PUT	`/api/rest/magazines/{id}`	Magazine	`200` Magazine	`404`
DELETE	`/api/rest/magazines/{id}`	—	`200` empty	`404`, `409`

`DiscMagEntity` extends `MagazineEntity`, so this list also contains disc magazines — a disc magazine *is* a magazine. Each carries `"productType": "DiscMagEntity"` and an extra `hasDisc` field.

1.3 Disc magazines — `/api/rest/discmags`

Verb	Path	Expects	Returns	Errors
GET	`/api/rest/discmags`	—	`200` array of DiscMag	—
GET	`/api/rest/discmags/{id}`	—	`200` DiscMag	`404`
POST	`/api/rest/discmags`	DiscMag	`200` DiscMag with generated id	—
PUT	`/api/rest/discmags/{id}`	DiscMag	`200` DiscMag	`404`
DELETE	`/api/rest/discmags/{id}`	—	`200` empty	`404`, `409`

The narrower view over the same rows: only products whose discriminator is `DISCMAG`.

1.4 Tickets — `/api/rest/tickets`

Verb	Path	Expects	Returns	Errors
GET	`/api/rest/tickets`	—	`200` array of Ticket	—
GET	`/api/rest/tickets/{id}`	—	`200` Ticket	`404`
POST	`/api/rest/tickets`	Ticket	`200` Ticket with generated id	—
PUT	`/api/rest/tickets/{id}`	Ticket	`200` Ticket	`404`
DELETE	`/api/rest/tickets/{id}`	—	`200` empty	`404`, `409`

2. Data Shapes

All four inherit `id`, `productId` and a read-only `productType` from `ProductEntity`. `id` is generated by the database and ignored on POST. `carts` is `@JsonIgnored` — it is the inverse side of the cart many-to-many and would otherwise recurse forever.

Book

```
{
  "id": 1,
  "productId": "SKU-1000",
  "productType": "BookEntity",
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "isbn": "9780000000001",
  "price": 15.99,
  "copies": 100
}
```

Magazine

```
{
  "id": 4,
  "productId": "SKU-1003",
  "productType": "MagazineEntity",
  "title": "Java Monthly",
  "price": 5.99,
  "copies": 100,
  "orderQty": 50,
  "currentIssue": "2026-11-02T10:08:01"
}
```

DiscMag

```
{
  "id": 5,
  "productType": "DiscMagEntity",
  "title": "eum DiscMag",
  "price": 24.99,
  "copies": 8,
  "orderQty": 30,
  "hasDisc": true
}
```

Ticket

```
{
  "id": 6,
  "productId": "SKU-1005",
  "productType": "TicketEntity",
  "description": "Spring Boot conference ticket",
  "price": 199.00
}
```

Error model

Status	When	Example body
404	No row with that id	`Could not find magazine with ID: 9999`
409	The product already appears on an order	`Could not delete product with ID: 4 - it already appears on 1 order(s)`
400	The JSON could not be parsed or bound	Spring's default body

Two `@RestControllerAdvice` classes produce these. `RestExceptionAdvice` handles `ResourceNotFoundException` — the base of the magazine, disc magazine and ticket exceptions — plus `ProductInUseException`; `BookNotFoundAdvice` handles the book one. The `409` exists because a product can sit in carts and on past orders: DELETE unlinks it from every cart, but refuses one that already appears on an order, since an order is a receipt and must not change.

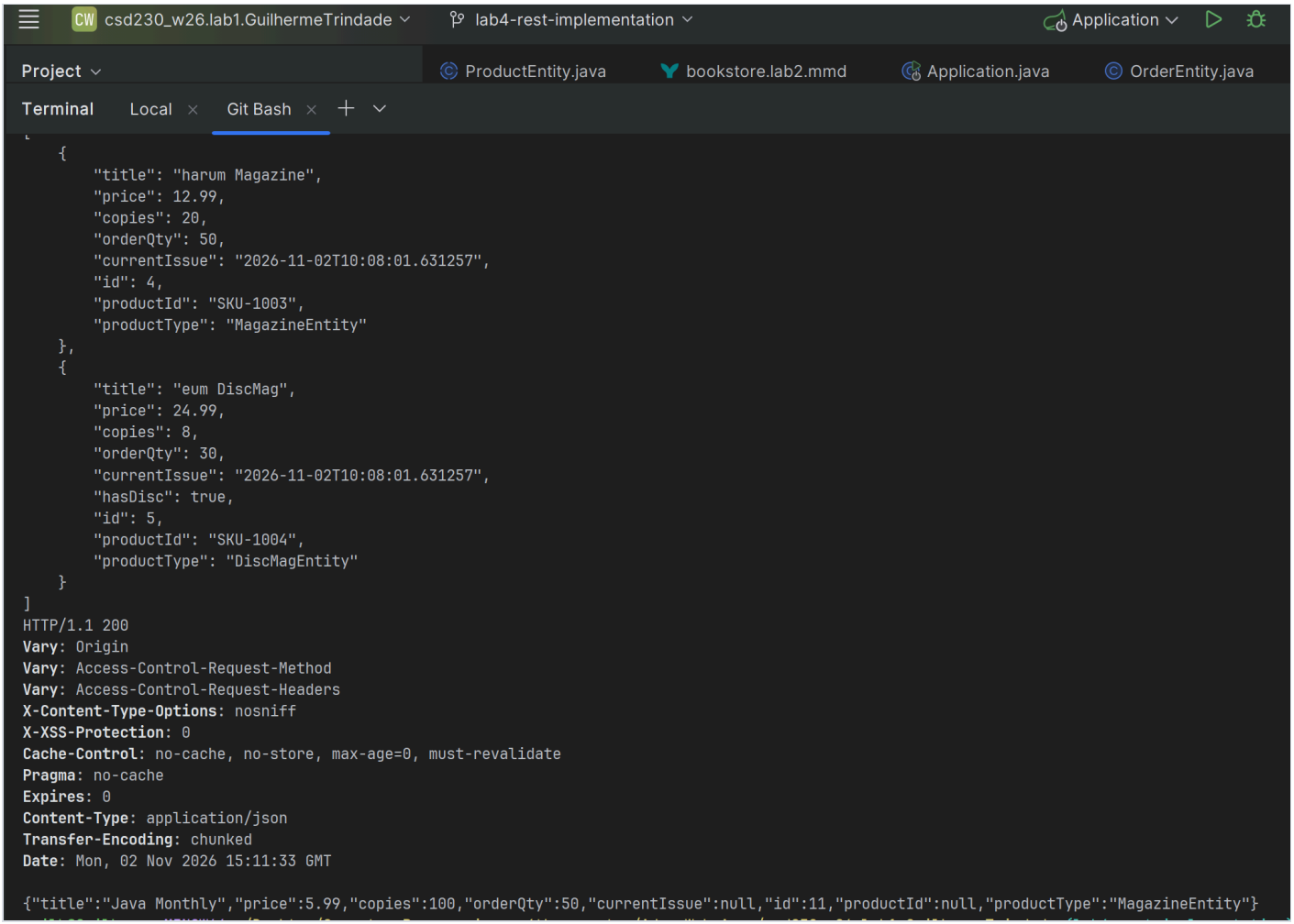
3. Testing with cURL

Run against the h2 profile (`./mvnw spring-boot:run -Dspring-boot.run.profiles=h2`). The `-i` flag prints the response headers, so the status line is visible in each result.

3.1 Magazine — read and create

```
curl -s localhost:8080/api/rest/magazines | python -m json.tool

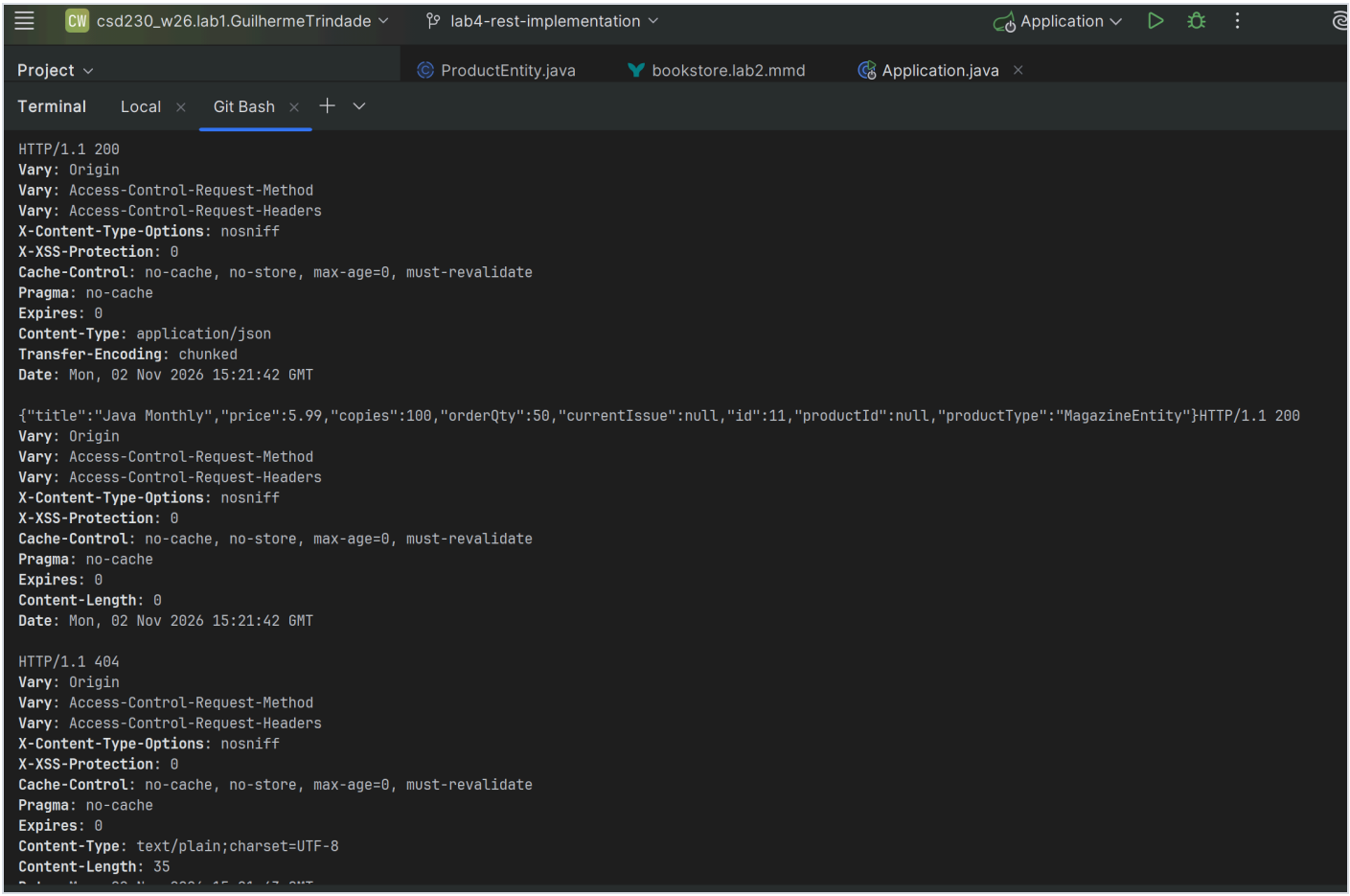
curl -i -X POST localhost:8080/api/rest/magazines \
  -H 'Content-Type: application/json' \
  -d '{"title":"Java Monthly","price":5.99,"copies":100,"orderQty":50}'
```



The list returns the seeded magazine (id 4) and disc magazine (id 5). The POST answers `HTTP/1.1 200` and returns the new magazine with generated id 11.

3.2 Magazine — read one, delete, confirm gone

```
curl -i localhost:8080/api/rest/magazines/11
curl -i -X DELETE localhost:8080/api/rest/magazines/11
curl -i localhost:8080/api/rest/magazines/11
```

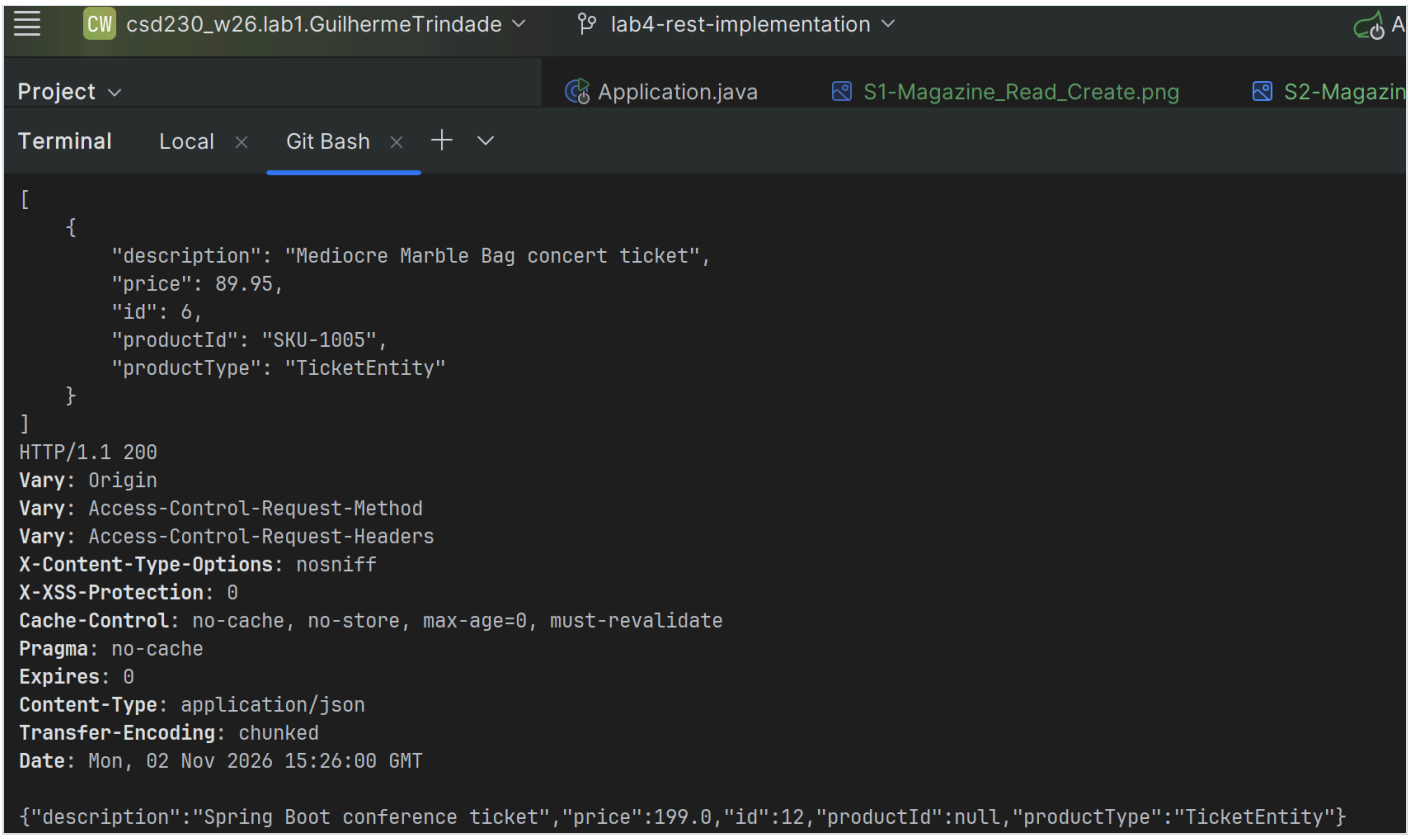


`200` with the body, `200` on the delete, then `404` with `Content-Type: text/plain` — the message produced by the exception advice, not a Whitelabel error page.

3.3 Ticket — read and create

```
curl -s localhost:8080/api/rest/tickets | python -m json.tool

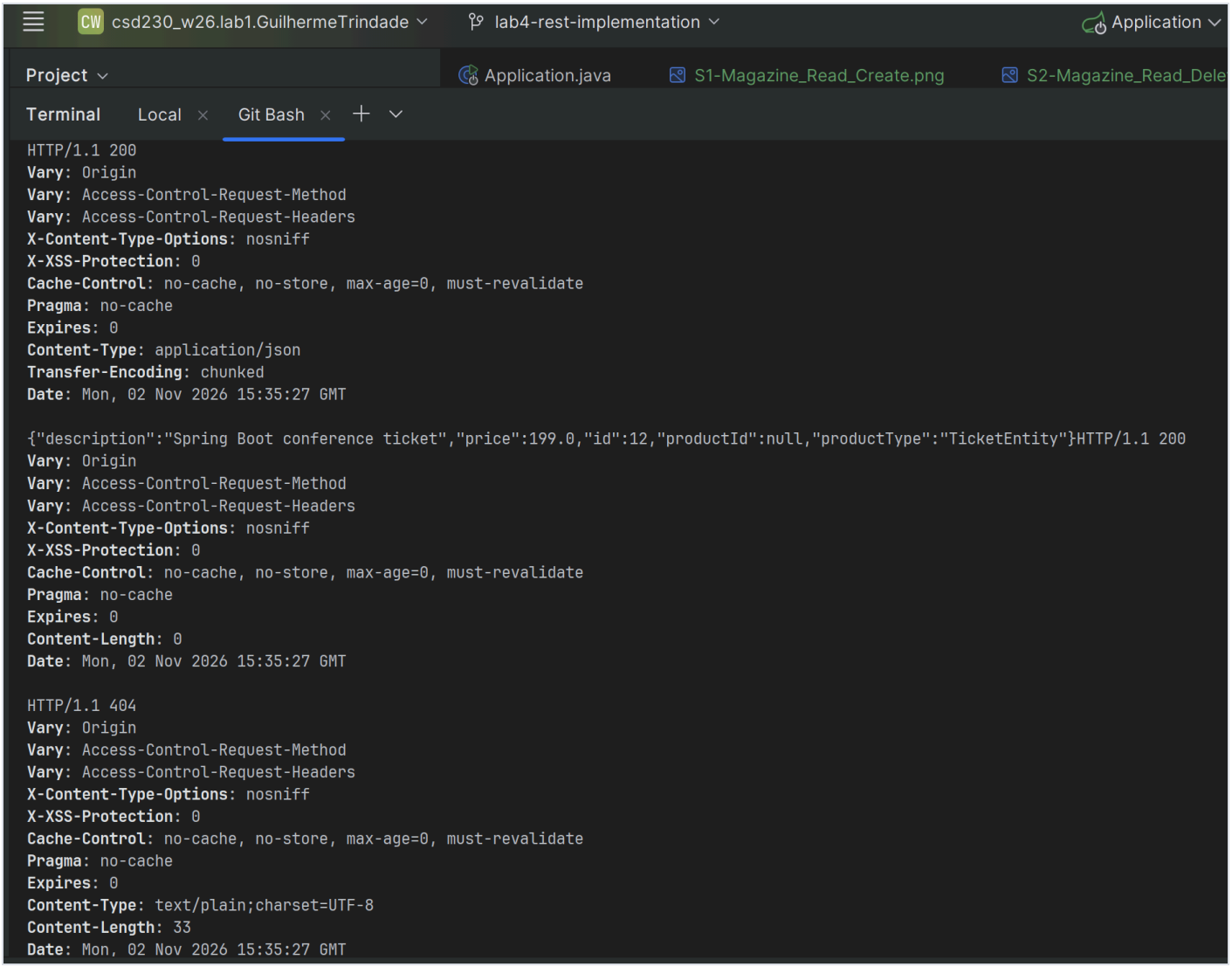
curl -i -X POST localhost:8080/api/rest/tickets \
  -H 'Content-Type: application/json' \
  -d '{"description":"Spring Boot conference ticket","price":199.00}'
```



A ticket has `description` and `price` instead of `title` and `copies`, so this exercises the other branch of the product hierarchy. The POST returns the new ticket with id 12.

3.4 Ticket — read one, delete, confirm gone

```
curl -i localhost:8080/api/rest/tickets/12
curl -i -X DELETE localhost:8080/api/rest/tickets/12
curl -i localhost:8080/api/rest/tickets/12
```

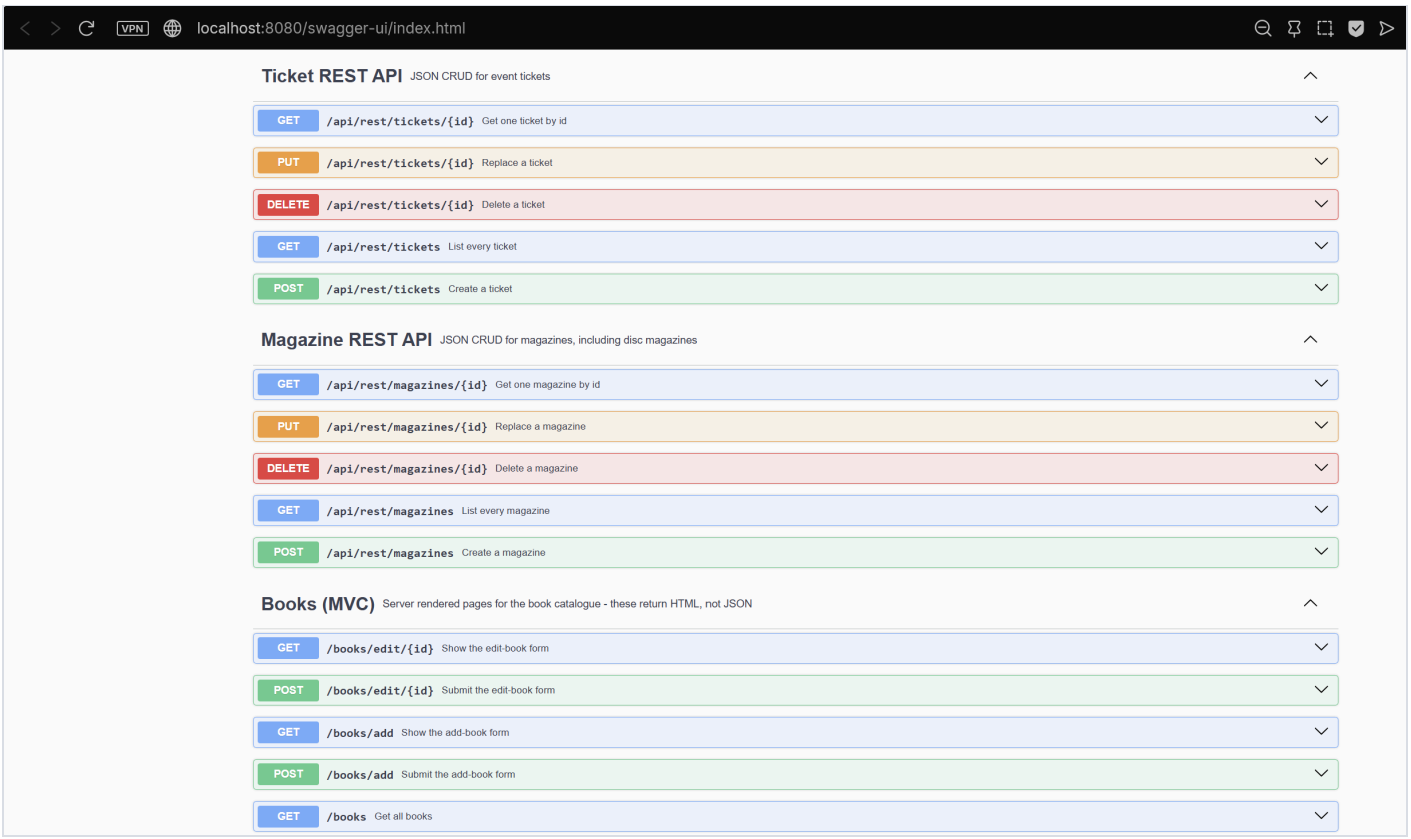


Same three-step proof for the second entity: read, delete, then `404`.

4. Swagger UI

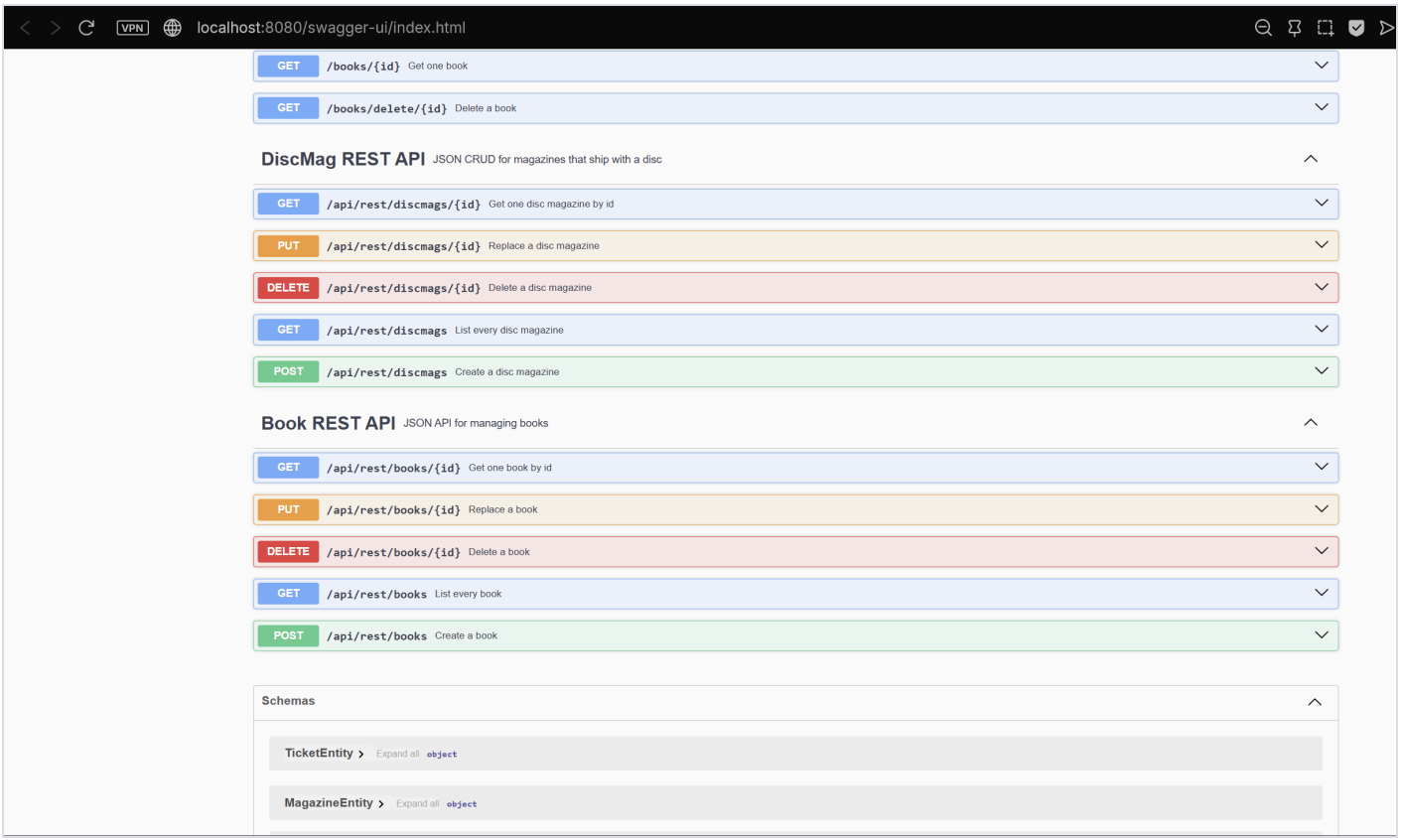
Served at `http://localhost:8080/swagger-ui/index.html` from the `springdoc-openapi` starter. Every controller carries a class-level `@Tag`, and every method an `@Operation` summary and one `@ApiResponse` per status it can return.

4.1 Tag groups (part 1 of 2)



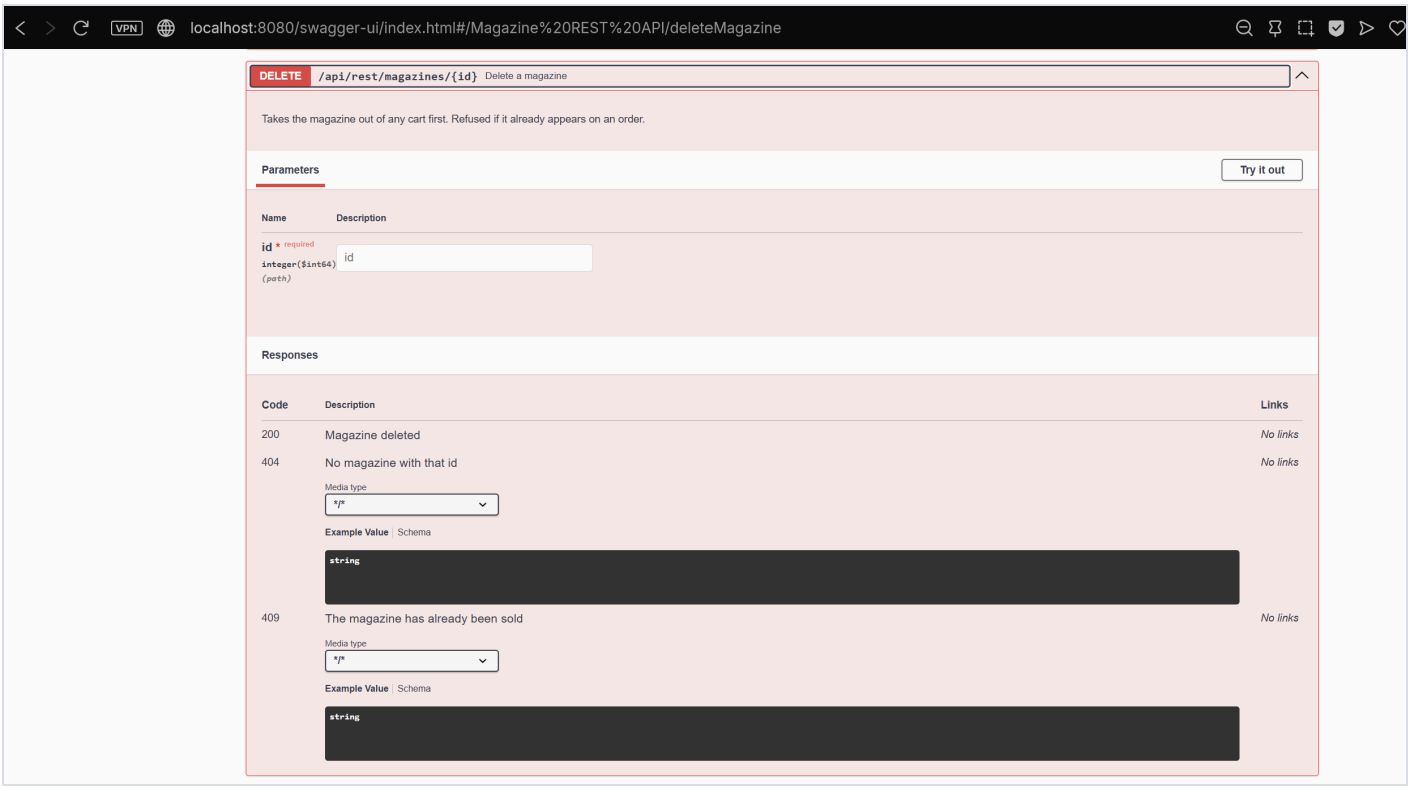
Ticket REST API and Magazine REST API, five operations each, followed by the MVC controller — grouped separately because those return HTML, not JSON.

4.2 Tag groups (part 2 of 2)



DiscMag REST API and Book REST API, plus the generated schema list. Twenty REST operations across the four resources.

4.3 One endpoint expanded



`DELETE /api/rest/magazines/{id}` expanded. The description comes from `@Operation`; the three rows under Responses are the three `@ApiResponse` annotations on the method.